

Deep Reinforcement Learning

Module 3: From DQN to DDQN and Policy-Based Methods

Mustafa Ozger

Aalborg University

March 10, 2026

Why this Module 3 is restructured

- In Module 2, the class covered the core idea of **DQN**, but **DDQN** remained for later.
- Therefore, this lecture begins with a **DQN refresh**, then introduces **DDQN**, and only then moves to the first **policy-based** ideas.
- This gives a smoother progression:

value-based deep RL $\longrightarrow$ improved value-based RL $\longrightarrow$ direct policy learning.

Pedagogical goal

Students should first consolidate *what DQN does*, then understand *what DDQN fixes*, and finally see *why we need policy-based methods at all*.

Where Module 3 fits in the course

- Module 1: MDPs, returns, value functions, Bellman equations, tabular RL.
- Module 2: Deep value-based RL with **DQN**.
- Module 3 (this lecture):
 - refresh **DQN**,
 - introduce **DDQN**,
 - start **policy-based methods** and **actor-critic thinking**.

Big picture

DQN and DDQN learn a *value function* and derive actions from it. Policy-based methods learn the *policy itself*.

After this lecture, students should be able to:

- recall the main building blocks of **DQN** and explain why replay and target networks are used,
- explain the **overestimation problem** in DQN,
- derive and interpret the **DDQN target**,
- compare **DQN** and **DDQN** conceptually,
- explain why value-based methods are not always enough,
- define a parameterized policy $\pi_{\theta}(a|s)$,
- explain the intuition of **REINFORCE**, baseline, advantage, and actor-critic.

Agenda

- ① DQN refresh: idea, architecture, replay, target network, loss
- ② DDQN: motivation, target, and why it is more stable
- ③ Why move beyond value-based methods?
- ④ Policy parameterization and policy objective
- ⑤ REINFORCE: the first policy-gradient method
- ⑥ Variance reduction: baseline and advantage
- ⑦ Actor–Critic as a bridge to later continuous-control methods

Notation used in this lecture

To keep the notation consistent across DQN, DDQN, and policy-gradient methods, we use:

s_t : current state, a_t : current action, r_{t+1} : reward after taking a_t ,
 s_{t+1} : next state, $\text{done}_t \in \{0, 1\}$: terminal indicator.

For the function approximators:

$Q_\theta(s, a)$: online Q-network, $Q_{\theta-}(s, a)$: target Q-network,
 $\pi_\theta(a|s)$: policy, $V_\phi(s)$: value function / critic.

Important note for fixed figures

Some imported figures use s' instead of s_{t+1} , and Q' instead of $Q_{\theta-}$. These mean the same objects in this lecture.

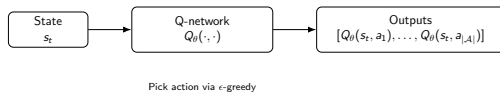
1) Core ideas: what DQN is trying to do

Goal: learn optimal action-values

$Q^*(s_t, a_t)$ = “how good is taking a_t in s_t and then acting optimally?”

Tabular Q-learning stores a table $Q(s_t, a_t)$.

- Works for small/discrete state spaces.
- Breaks for large/continuous/high-dimensional states (e.g., images).



DQN replaces the table with a neural network:

$$Q_\theta(s_t, a_t) \approx Q^*(s_t, a_t)$$

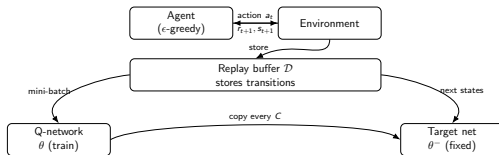
Network input: state s_t output: **Q-values for all actions.**

Key point: DQN learns the same Q-learning target, but uses a deep network to *generalize* across many states.

2) High-level DQN workflow (what happens repeatedly)

Loop (per environment step):

- 1 **Act:** choose a_t using Q-network and ϵ -greedy.
- 2 **Observe:** get $(r_{t+1}, s_{t+1}, \text{done}_t)$.
- 3 **Store:** push $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$ into replay buffer $\mathcal{D}$.
- 4 **Learn:** sample a random mini-batch from $\mathcal{D}$ and update the Q-network.
- 5 **Sync:** every C steps, copy weights $\theta^- \leftarrow \theta$.



Takeaway

DQN alternates between **data collection** (act/observe/store) and **supervised-style regression** (learn from batches).

3) Q-network predicts the *Predicted Q-value* (current state)

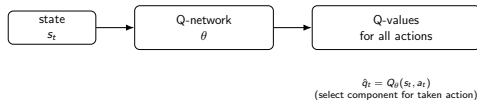
For a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$:

Forward pass on current state:

$$[Q_\theta(s_t, a_1), \dots, Q_\theta(s_t, a_{|\mathcal{A}|})]$$

Predicted Q-value for the action actually taken:

$$\hat{q}_t = Q_\theta(s_t, a_t)$$



During interaction (acting), we choose a_t via ϵ -greedy:

$$a_t = \begin{cases} \text{random} & \text{w.p. } \epsilon \\ \arg \max_a Q_\theta(s_t, a) & \text{w.p. } 1 - \epsilon \end{cases}$$

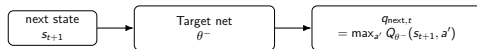
Takeaway

The Q-network provides action preferences; exploration (random choice) ensures we don't get stuck too early.

4) Target network predicts the *Target Q-value* (next state)

For the same transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$:

Forward pass on next state with the Target network:



$$q_{\text{next},t} = \max_{a'} Q_{\theta^-}(s_{t+1}, a')$$

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t) q_{\text{next},t}$$

Build the TD target (terminal-aware):

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t) q_{\text{next},t}$$

- If $\text{done}_t = 1$: $y_t = r_{t+1}$ (no bootstrap beyond terminal).
- Else: bootstrap from the best estimated next action.

Takeaway

The Target network provides a **stable** bootstrap value so the training target doesn't shift every gradient step.

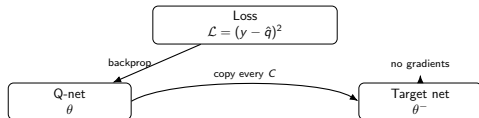
5) Compute loss and train the Q-network (backprop only here)

Predicted: $\hat{q}_t = Q_\theta(s_t, a_t)$ **Target:** $y_t = r_{t+1} + \gamma(1 - \text{done}_t) \max_{a'} Q_{\theta^-}(s_{t+1}, a')$

Loss (MSE) on a mini-batch of size B :

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{i=1}^B (y_i - Q_\theta(s_i, a_i))^2$$

- We **backpropagate** $\nabla_\theta \mathcal{L}(\theta)$ and update **only** θ .
- The Target network parameters θ^- are **frozen** during this update.
- Every C steps, we do a hard update: $\theta^- \leftarrow \theta$.



Takeaway

Training looks like supervised regression: make the Q-network's output match a (temporarily) stable TD target.

How do SGD and Adam update the network weights?

After backpropagation, we obtain the gradient of the loss with respect to the parameters:

$$\nabla_{\theta} \mathcal{L}(\theta).$$

An optimizer uses this gradient to update the weights.

SGD: Stochastic Gradient Descent

$$\theta \leftarrow \theta - \eta \nabla_{\theta} \mathcal{L}(\theta)$$

- η is the learning rate.
- Move parameters in the direction that reduces the loss.
- Simple and intuitive.
- Same step-size idea for all parameters.

Intuition

If a weight increases the loss, SGD pushes it down. If a weight decreases the loss, SGD pushes it up.

Adam: Adaptive Moment Estimation

Adam also uses gradients, but keeps running estimates of:

- the **mean** of gradients,
- the **variance / magnitude** of gradients.

This allows Adam to use **adaptive step sizes** for different parameters.

$$\theta \leftarrow \theta - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- Often converges faster in deep learning.
- Usually more robust when gradients are noisy.
- Very common in DQN and policy-gradient implementations.

6) Why Replay + Why Target net? (and DQN operation in depth)

Why do we need Experience Replay?

- Consecutive transitions are **correlated** (bad for SGD).
- Replay makes training batches more **i.i.d.-like** by random sampling.
- Reuses old experience $\Rightarrow$ better sample efficiency.

Why do we need a Target network?

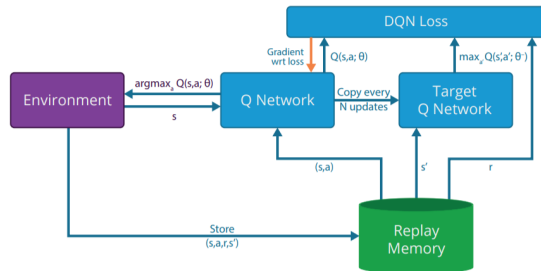
- Without it, the target y_t changes immediately after each update (**moving target**).
- Freezing θ^- stabilizes bootstrapping and reduces oscillations/divergence.

Key point: Replay tackles **correlated data**; the Target net tackles the **moving-target** problem.

DQN operation in depth (one training iteration)

- 1 Collect $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$ and store in $\mathcal{D}$.
- 2 Sample random batch from $\mathcal{D}$.
- 3 Compute $\hat{q}_i = Q_\theta(s_i, a_i)$ (Q-net).
- 4 Compute $y_i = r_i + \gamma(1 - d_i) \max_{a'} Q_{\theta^-}(s'_i, a')$ (Target net).
- 5 Compute $\mathcal{L}(\theta)$ and **backprop only to θ** .
- 6 Every C steps: $\theta^- \leftarrow \theta$.

DQN architecture: putting the pieces together



DQN system diagram (provided figure).

Notation note: in this figure, s and s' correspond to s_t and s_{t+1} in the lecture notation; the online and target networks correspond to Q_θ and Q_{θ^-} , respectively.

Key message

Replay buffer and target network are the two main engineering ideas that make DQN much more stable than naive deep Q-learning.

What DQN still struggles with

Even with replay and target networks, DQN still has important limitations:

- Q-values can be **overestimated** because of the max operator.
- It is designed mainly for **discrete action spaces**.
- For continuous actions, solving

$$\arg \max_a Q(s, a)$$

at every step is difficult.

- If we want a naturally stochastic policy, DQN is indirect: it learns values first and policy second.

Transition

DDQN addresses the first issue. Policy-based methods address the later ones.

Why the max operator can overestimate

Suppose the true next-state action values are close, but our estimates contain noise.

Action	True value	Noisy estimate
a_1	5.0	5.8
a_2	5.0	4.9

Then

$$\max_{a'} Q(s', a') = 5.8$$

selects the positively biased estimate.

Key intuition

The max operator prefers large noisy estimates, so it can systematically push targets upward.

DDQN: main idea

- In DQN, the same target network both:
 - selects the best next action,
 - evaluates that action.
- In **Double DQN (DDQN)**, we separate these two roles.

Decoupling principle

- **Selection:** use the *online* network to choose the best next action.
- **Evaluation:** use the *target* network to evaluate that chosen action.

Why this helps

It reduces the chance that the same noisy estimator both picks and confirms an overly optimistic action.

DQN target vs DDQN target

DQN target

$$y_t^{\text{DQN}} = r_{t+1} + \gamma(1 - \text{done}_t) \max_{a'} Q_{\theta-}(s_{t+1}, a')$$

Same target network does both:

- action selection,
- action evaluation.

DDQN target

$$y_t^{\text{DDQN}} = r_{t+1} + \gamma(1 - \text{done}_t) Q_{\theta-}(s_{t+1}, \arg \max_{a'} Q_{\theta}(s_{t+1}, a'))$$

Different roles:

- online network selects,
- target network evaluates.

What changed?

The improvement is entirely in the **target computation**; the rest of the DQN pipeline stays almost the same.

DDQN target step by step

For a sampled transition $(s_t, a_t, r_{t+1}, s_{t+1}, \text{done}_t)$:

- 1 Use the online network to choose the greedy next action:

$$a_t^* = \arg \max_{a'} Q_{\theta}(s_{t+1}, a').$$

- 2 Use the target network to evaluate that action:

$$Q_{\theta-}(s_{t+1}, a_t^*).$$

- 3 Form the target:

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t)Q_{\theta-}(s_{t+1}, a_t^*).$$

Practical conclusion

DDQN looks almost identical to DQN in code; only one line in the target changes, but the effect on stability can be important.

DQN and DDQN side by side

Aspect	DQN	DDQN
Target	$r + \gamma \max_{a'} Q_{\theta-}(s', a')$	$r + \gamma Q_{\theta-}(s', \arg \max_{a'} Q_{\theta}(s', a'))$
Action selection in target	Target network	Online network
Action evaluation in target	Target network	Target network
Main issue addressed	Basic instability	Overestimation bias
Replay + target nets	Yes	Yes

Teaching point

DDQN is not a different family of RL. It is a **refinement of DQN** that keeps the same general workflow.

DDQN workflow in words

- 1 Collect experience with ε -greedy interaction.
- 2 Store transitions in replay.
- 3 Sample a minibatch.
- 4 Compute next action with online network:

$$a_t^* = \arg \max_{a'} Q_{\theta}(s_{t+1}, a').$$

- 5 Evaluate that action with target network.
- 6 Update the online network using the DDQN target.
- 7 Periodically refresh the target network.

What stays the same?

Replay buffer, target network, minibatch training, and epsilon-greedy exploration are all still there.

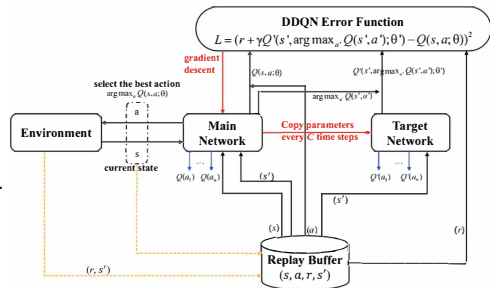
DDQN architecture: components and data flow

- **Act (main / online net):** choose a via ϵ -greedy from $Q(s, \cdot; \theta)$.
- **Store:** save $(s, a, r, s', \text{done})$ in the replay buffer.
- **Sample:** draw a minibatch from replay (decorrelates experience).
- **DDQN target (selection vs evaluation):**

$$a^* = \arg \max_{a'} Q(s', a'; \theta), \quad y = r + (1 - \text{done}) \gamma Q'(s', a^*; \theta^-).$$

- **Optimize:** $\mathcal{L}(\theta) = (y - Q(s, a; \theta))^2 \Rightarrow$ gradients update θ only.
- **Sync target:** every C steps, $\theta^- \leftarrow \theta$.

Notation note: the imported figure uses s' for the next state, so $s' \equiv s_{t+1}$. It may also write the target-network evaluation as $Q'(s', a^*; \theta^-)$; in our lecture notation this is exactly $Q_{\theta^-}(s_{t+1}, a_t^*)$. Likewise, $Q(s, a; \theta)$ in the figure is the online network $Q_{\theta}(s_t, a_t)$.



What DDQN fixes – and what it does not

DDQN improves

- overestimation of Q-values,
- training stability in many tasks,
- reliability of the Bellman target.

DDQN does not solve

- continuous-action optimization,
- direct learning of stochastic policies,
- the general sample inefficiency of trial-and-error RL,
- all instability caused by function approximation.

Natural next question

If even improved Q-learning is still awkward for some tasks, should we learn the *policy directly*?

Bridge to policy-based methods

- DQN/DDQN learn a score for each action, then choose the best action.
- This is natural when the number of actions is small and discrete.
- But in many tasks we want:
 - continuous actions,
 - directly stochastic behavior,
 - a parameterized controller instead of an implicit greedy rule.

New viewpoint

Instead of learning $Q(s, a)$ and then deriving the policy, we can learn the policy itself.

Value-based vs policy-based methods

Value-based

- Learn $Q(s, a)$ or $V(s)$
- Policy is derived indirectly
- Typical action rule:

$$a = \arg \max_a Q(s, a)$$

- Best suited to discrete actions

Policy-based

- Learn $\pi_{\theta}(a|s)$ directly
- Policy is the primary object
- Actions come from sampling or deterministic output
- Natural for continuous control and stochastic policies

What is a policy?

A **policy** is a rule for choosing actions given a state.

Two common forms

- **Stochastic policy:**

$$\pi_{\theta}(a|s) = \Pr(A = a \mid S = s)$$

- **Deterministic policy:**

$$a = \mu_{\theta}(s)$$

Control interpretation

A policy is the controller itself: from state or observation, it directly outputs a decision.

Policy parameterization: common choices

- For **discrete actions**, use a softmax policy:

$$\pi_{\theta}(a|s) = \frac{\exp(z_{\theta}(s)_a)}{\sum_k \exp(z_{\theta}(s)_k)}.$$

- For **continuous actions**, use a Gaussian policy:

$$a \sim \mathcal{N}(\mu_{\theta}(s), \sigma_{\theta}(s)^2).$$

- A neural network outputs logits, means, variances, or actions directly.

Key shift from DQN

The network no longer predicts only action values. It parameterizes how the agent actually behaves.

Objective: what do we optimize?

For a policy π_θ , define the expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^{T-1} \gamma^t r_{t+1} \right],$$

where $\tau = (s_0, a_0, r_1, s_1, \dots)$ is a trajectory.

- $J(\theta)$ is a single number measuring policy quality.
- Learning means changing θ to increase $J(\theta)$.

Main question

How can we compute a gradient of expected return when the environment is a black box?

Policy gradient intuition

- If an action produced high return, we want to make that action **more likely** in similar states.
- If an action produced poor return, we want to make that action **less likely**.
- This is the conceptual meaning of a policy gradient update.

Learning rule in words

Increase the probability of actions that led to good outcomes; decrease the probability of actions that led to bad outcomes.

The log-derivative trick

A key identity is:

$$\nabla_{\theta} \pi_{\theta}(a|s) = \pi_{\theta}(a|s) \nabla_{\theta} \log \pi_{\theta}(a|s).$$

Why it matters

It lets us write gradients of expectations in terms of $\nabla_{\theta} \log \pi_{\theta}(a|s)$, which we can compute from sampled trajectories.

Important intuition

We do not need to differentiate through the environment dynamics. We only need gradients of the policy.

REINFORCE: the simplest policy-gradient method

REINFORCE uses the Monte Carlo return G_t :

$$G_t = \sum_{k=t}^{T-1} \gamma^{k-t} r_{k+1}.$$

Then the gradient estimate is:

$$\nabla_{\theta} J(\theta) \approx \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t.$$

Parameter update

$$\theta \leftarrow \theta + \alpha \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) G_t.$$

REINFORCE implementation checklist

① Run the policy for one episode and collect (s_t, a_t, r_{t+1}) .

② Compute returns backward:

$$G_t = r_{t+1} + \gamma G_{t+1}.$$

③ Form the actor loss:

$$\mathcal{L}_{\text{actor}} = - \sum_t \log \pi_{\theta}(a_t | s_t) G_t.$$

④ Backpropagate and update θ .

Practical note

This is conceptually simple, but the learning signal can be very noisy.

Main weakness of REINFORCE: high variance

- The return G_t depends on many random future events.
- Small differences in trajectories can create large differences in updates.
- Learning can therefore be unstable or slow.

Bias-variance view

Monte Carlo policy gradients are often **unbiased**, but they can have very **high variance**.

Baseline trick: reduce variance

We can subtract a baseline $b(s_t)$ without changing the expected gradient:

$$\nabla_{\theta} J(\theta) \approx \sum_t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) (G_t - b(s_t)).$$

Intuition

Instead of asking “was the total return large?”, we ask “was it larger or smaller than expected for this state?”

Typical choice

Use $b(s_t) \approx V(s_t)$, the state-value function.

Advantage function: a cleaner signal

Define the advantage as:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s).$$

- $A(s, a) > 0$: action is better than the average expectation at state s .
- $A(s, a) < 0$: action is worse than expected.

Policy update with advantage

$$\nabla_\theta J(\theta) \approx \sum_t \nabla_\theta \log \pi_\theta(a_t | s_t) A_t.$$

From REINFORCE to Actor–Critic

- REINFORCE uses full returns G_t .
- A better idea is to learn a separate network that estimates value.
- This learned estimator is called the **critic**.

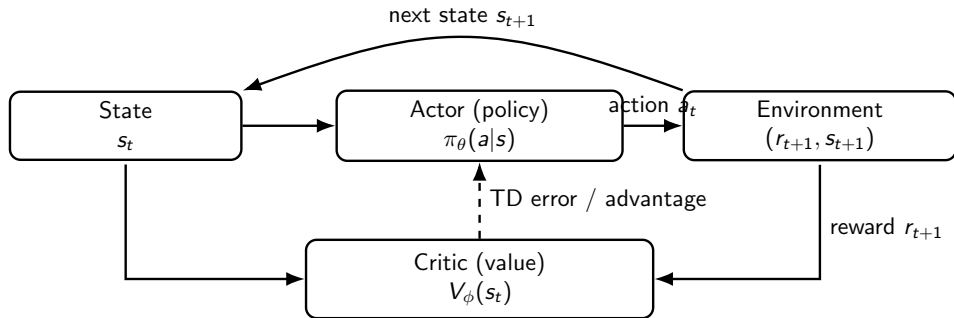
Roles

- **Actor**: policy $\pi_{\theta}(a|s)$
- **Critic**: value estimator $V_{\phi}(s)$ or $Q_{\phi}(s, a)$

Connection to earlier modules

The critic brings back value-function learning, but now it supports direct policy learning instead of replacing it.

Actor–Critic architecture



Key message

The critic converts experience into a learning signal that tells the actor whether its action was better or worse than expected.

A simple actor–critic update

For a value critic, a common TD target is:

$$y_t = r_{t+1} + \gamma(1 - \text{done}_t)V_\phi(s_{t+1}).$$

The TD error is:

$$\delta_t = y_t - V_\phi(s_t).$$

Critic update

Minimize

$$\mathcal{L}(\phi) = \mathbb{E}[(y_t - V_\phi(s_t))^2].$$

Actor update

$$\theta \leftarrow \theta + \alpha \nabla_\theta \log \pi_\theta(a_t | s_t) \delta_t.$$

On-policy vs off-policy: early preview

On-policy methods

- Learn from data generated by the same policy being updated.
- Examples: REINFORCE, simple actor–critic.
- Usually easier to reason about, but less sample-efficient.

Off-policy methods

- Learn a target policy from data generated by another behavior policy.
- DQN and DDQN are off-policy because they can learn from replayed past experience.
- Later actor–critic methods can also be off-policy.

Conceptual map of this lecture



- DQN: deep value approximation
- DDQN: better target estimation
- Policy gradients: direct policy learning
- Actor-Critic: policy learning supported by value estimation

- **DQN** uses a neural Q-network, replay buffer, and target network.
- **DDQN** modifies the target so that action selection and evaluation are separated.
- DDQN reduces **overestimation bias**, but it is still mainly a value-based method for discrete actions.
- **Policy-based methods** learn the policy directly instead of deriving it from Q-values.
- **REINFORCE** is the simplest policy-gradient method, but it has high variance.
- **Baseline, advantage, and actor–critic** improve the learning signal and prepare us for more advanced policy-learning methods.

What can follow next?

Depending on the pace of the course, the next part of Module 3 can continue with:

- deeper actor–critic implementations,
- deterministic policy gradients and DDPG,
- or a more stable modern policy method such as PPO.

Course continuity

Today we built the conceptual bridge from *improved Q-learning* to *direct policy learning*. That bridge is essential before continuous-control algorithms make sense.